JAVA INTERVIEW PREPARATION

CHAPTER 21

DATA STRUCTURES, COMPLEXITY, AND GRAPHS

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Practical Data Structures and Complexity in Java: Choosing Shapes That Match the Workload

Senior / Lead Java Developer Reading Chapter

Data-structure interviews are not tests of memorized Big-O tables alone. At senior level, the interviewer expects a candidate to connect access patterns, memory representation, concurrency, domain invariants, and operational scale. The right structure makes the common operation cheap and the invalid state difficult to represent.

Begin with Operations, Not Class Names

Before selecting a collection, write down the dominant operations and constraints:

- Lookup by identifier or by position?
- Preserve insertion order, sorted order, or no order?
- Insert at one end, both ends, or arbitrary positions?
- Retrieve the smallest or highest-priority item repeatedly?
- Need range queries, prefix queries, or graph traversal?
- What is the maximum cardinality and memory budget?
- Is mutation single-threaded, externally locked, or concurrent?

```
workload question                   likely starting point
lookup by unique key                HashMap
membership test                     HashSet
indexed iteration                   ArrayList
FIFO or double-ended operations     ArrayDeque
repeated minimum/maximum            PriorityQueue
sorted keys and range navigation    TreeMap / TreeSet
enum flags                          EnumSet
```

This is a starting point, not a replacement for measurement or domain analysis.

Complexity Is a Growth Model

Big-O describes how cost grows as input grows, usually ignoring constants and hardware effects.

```
O(1)       cost does not grow with n in the model
O(log n)   problem shrinks by a constant factor each step
O(n)       work grows proportionally with elements
O(n log n) efficient comparison sorting and divide-and-conquer pattern
O(n^2)     pairwise or nested full scans
```

Expected O(1) hash lookup can lose to O(log n) tree lookup for tiny collections because of constants. An O(n) array scan can be faster than pointer-heavy traversal due to locality. Complexity predicts scaling; benchmarking and profiling quantify a real implementation.

Arrays and Dynamic Arrays

An array provides fixed-size contiguous storage and O(1) indexed access. Primitive arrays pack values efficiently. Object arrays store references.

```
int[] latencyBuckets = new int[100];
latencyBuckets[42]++;
```

ArrayList adds dynamic size and collection APIs around an Object array. It is usually the best general list, but primitive boxing and middle shifts remain relevant. Arrays are useful for fixed dimensions, performance-sensitive primitives, lookup tables, heaps, and adjacency structures when the index space is dense.

Stacks, Queues, and Deques

ArrayDeque implements a resizable circular buffer and efficiently adds or removes at both ends.

```
backing circular array
       head -> [B][C][D][_][_][A] <- tail wraps
logical order: A, B, C, D
```

Use it as a stack:

```
Deque<Node> stack = new ArrayDeque<>();
stack.push(root);
Node next = stack.pop();
```

Or as a FIFO queue:

```
Deque<Job> queue = new ArrayDeque<>();
queue.addLast(job);
Job next = queue.removeFirst();
```

Prefer Deque to the legacy Stack class. ArrayDeque rejects null, which keeps methods such as poll returning null unambiguous. It is not thread-safe; use BlockingQueue or ConcurrentLinkedDeque when the concurrency contract requires them.

Priority Queues and Heaps

PriorityQueue is normally a binary heap stored in an array. It provides O(1) access to the head, O(log n) insertion, and O(log n) removal of the head.

```
heap logical tree          array representation
        2                  [2, 5, 4, 9, 8, 7]
      /   \
     5     4
    / \   /
   9   8 7
```

```
PriorityQueue<Job> ready = new PriorityQueue<>(
    Comparator.comparing(Job::deadline)
              .thenComparing(Job::id)
);

ready.add(job);
Job mostUrgent = ready.poll();
```

Iteration is not sorted. Only repeated poll returns priority order. Removing an arbitrary element is generally linear because the heap does not maintain a key-to-index lookup.

PriorityQueue is not a scheduler by itself. It does not wait until a future deadline, provide persistence, or coordinate multiple service instances. Java's DelayQueue or a durable scheduling system may match those requirements better.

Hash Tables and Trees

HashMap offers expected O(1) key lookup without ordering. TreeMap uses a balanced search tree with O(log n) lookup and sorted navigation.

```
NavigableMap<Instant, Event> timeline = new TreeMap<>();
Map.Entry<Instant, Event> next = timeline.ceilingEntry(now);
NavigableMap<Instant, Event> hour =
    timeline.subMap(start, true, end, false);
```

Choose a tree when sorted iteration, nearest-neighbor operations, or ranges are primary. Do not sort an entire HashMap on every request if a continuously sorted index is what the application needs. Conversely, do not pay tree costs only to perform exact key lookup.

Building an LRU Cache

LinkedHashMap can maintain access order and evict the eldest entry.

```
final class LruCache<K, V> extends LinkedHashMap<K, V> {
    private final int maximumSize;

    LruCache(int maximumSize) {
        super(16, 0.75f, true); // access order
        this.maximumSize = maximumSize;
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > maximumSize;
    }
}
```

This illustrates combining a hash table for lookup with a linked order for eviction. It is not automatically thread-safe and lacks production cache features such as time expiration, weight limits, refresh, statistics, asynchronous loading, and stampede control. Prefer a mature cache library for serious caching, but understand the underlying composition.

Graph Representation

A graph models entities connected by edges. An adjacency list is usually efficient for sparse graphs.

```
Map<String, List<String>> adjacency = new HashMap<>();
adjacency.computeIfAbsent("A", ignored -> new ArrayList<>()).add("B");
adjacency.computeIfAbsent("A", ignored -> new ArrayList<>()).add("C");
```

```
A -> B, C
B -> D
C -> D
D ->
```

An adjacency matrix uses O(V squared) space but offers constant-time edge checks and can be appropriate for small dense graphs. Representation should follow graph density and operations.

Graph Vocabulary and the Shape of the Problem

A vertex represents an entity and an edge represents a relationship. Before choosing an algorithm, classify the relationship because the classification changes what paths and invariants mean.

```
undirected social connection          directed dependency

     A -------- B                     service-A ------> service-B
      \        /                                        |
       \      /                                         v
         C                                          database-C

weighted route                          directed acyclic graph (DAG)

     A --5--> B                           compile ----> test ----> deploy
      \       |
       2      1
        \     v
          --> C
```

In an undirected graph, an edge connects both ways. In a directed graph, `A -> B` does not imply `B -> A`. Weighted edges carry cost such as distance, latency, or price. A cycle is a path that returns to an earlier vertex. A DAG has directed edges and no cycles, enabling dependency ordering.

Also clarify whether parallel edges and self-loops are allowed. A `Set` of neighbors removes duplicate edges, while a `List` preserves duplicates and encounter order. That choice is part of the domain model, not a minor implementation detail.

Adjacency List versus Adjacency Matrix

Let `V` be the number of vertices and `E` the number of edges.

```
representation       space        edge test       iterate neighbors
-------------------------------------------------------------------
adjacency list       O(V + E)     O(degree)       O(degree)
adjacency hash sets  O(V + E)     expected O(1)   O(degree)
adjacency matrix     O(V^2)       O(1)            O(V)
edge list            O(E)         O(E)            O(E) without index
```

Sparse application graphs usually favor adjacency lists because most possible vertex pairs have no edge. A matrix can fit a small dense graph or an algorithm dominated by repeated edge-existence checks. An edge list is natural for batch algorithms such as Kruskal's minimum spanning tree, but traversal usually needs an index.

```
same graph as adjacency list          adjacency matrix

A: B, C                                  to: A B C D
B: D                                  A      0 1 1 0
C: D                                  B      0 0 0 1
D:                                    C      0 0 0 1
                                      D      0 0 0 0
```

For undirected graphs, add both directions or design an edge abstraction that makes symmetry explicit. Forgetting the reverse edge is a common source of incomplete traversal.

A Typed Java Graph Model

A raw `Map<String, List<String>>` teaches traversal, but production code benefits from explicit ownership and validation.

```
record Edge<V>(V to, int weight) {}

final class DirectedGraph<V> {
    private final Map<V, List<Edge<V>>> outgoing = new HashMap<>();

    void addVertex(V vertex) {
        outgoing.computeIfAbsent(vertex, ignored -> new ArrayList<>());
    }

    void addEdge(V from, V to, int weight) {
        if (weight < 0) {
            throw new IllegalArgumentException("negative weight");
        }
        addVertex(from);
        addVertex(to);
        outgoing.get(from).add(new Edge<>(to, weight));
    }

    List<Edge<V>> outgoingFrom(V vertex) {
        return List.copyOf(outgoing.getOrDefault(vertex, List.of()));
    }
}
```

The class makes direction explicit, records vertices with no outgoing edges, rejects unsupported weights, and does not expose its mutable adjacency lists. For high-volume graphs, copying every neighbor list may be expensive; an immutable built graph, read-only view, primitive IDs, compressed adjacency arrays, or database-backed traversal may be more appropriate.

Vertex equality determines graph identity when vertices are map keys. Mutable vertices can corrupt adjacency lookup in the same way as mutable HashMap keys. Stable IDs or immutable value objects are safer.

Breadth-First Search

Breadth-first search uses a queue and visits nodes by distance in an unweighted graph. A visited set prevents cycles and repeated work.

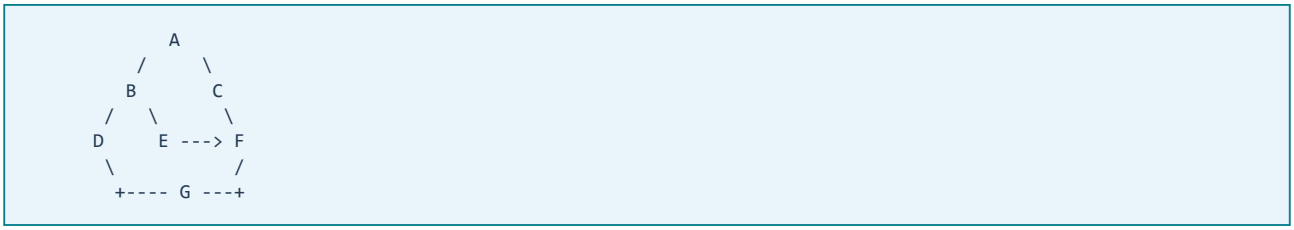
```
List<String> breadthFirst(Map<String, List<String>> graph, String start) {
    List<String> order = new ArrayList<>();
    Set<String> visited = new HashSet<>();
    Deque<String> queue = new ArrayDeque<>();

    visited.add(start);
    queue.addLast(start);

    while (!queue.isEmpty()) {
        String node = queue.removeFirst();
        order.add(node);

        for (String neighbour : graph.getOrDefault(node, List.of())) {
            if (visited.add(neighbour)) {
                queue.addLast(neighbour);
            }
        }
    }
    return order;
}
```

With adjacency lists, time is O(V + E) because each vertex and edge is processed a bounded number of times. Space is O(V) beyond the graph for queue, visited set, and output.

Mark nodes visited when enqueuing, not when dequeuing, to avoid adding the same node many times through different incoming edges.

BFS as a Layer-by-Layer Memory Image

Consider this unweighted directed graph:



BFS expands the frontier in distance layers. The queue holds discovered vertices whose neighbors have not yet been processed.

step	remove	queue after discovery	visited / distance
0	-	[A]	A:0
1	A	[B, C]	B:1, C:1
2	B	[C, D, E]	D:2, E:2
3	C	[D, E, F]	F:2
4	D	[E, F, G]	G:3
5	E	[F, G]	no new vertex
6	F	[G]	no new vertex
7	G	[]	complete

This queue picture explains the shortest-path property: in an unweighted graph, the first time a vertex is discovered, BFS has found a path using the minimum number of edges from the start. It does not minimize arbitrary edge weights.

Reconstructing an Unweighted Shortest Path

Store the predecessor that first discovered each vertex:

```
Map<String, String> parent = new HashMap<>();
Deque<String> queue = new ArrayDeque<>();
Set<String> visited = new HashSet<>();

visited.add(start);
queue.addLast(start);

while (!queue.isEmpty()) {
    String current = queue.removeFirst();
    if (current.equals(target)) break;

    for (String next : graph.getOrDefault(current, List.of())) {
        if (visited.add(next)) {
            parent.put(next, current);
            queue.addLast(next);
        }
    }
}
```

Then walk backward from target to start and reverse the result. Distinguish “target equals start,” “target unreachable,” and “target absent from the representation.” Production APIs should return an explicit result rather than an empty list whose meaning is ambiguous.

```
discovered parents:  G <- D <- B <- A
reverse:              A -> B -> D -> G
```

Depth-First Search and Recursion Risk

Depth-first search uses a stack, explicit or recursive. It supports reachability, cycle detection, topological algorithms, and component discovery.

Recursive code is concise, but a very deep graph can overflow the Java thread stack. An explicit ArrayDeque makes depth a heap-backed data-structure concern and gives more control.

```
Deque<String> stack = new ArrayDeque<>();
stack.push(start);
while (!stack.isEmpty()) {
    String node = stack.pop();
    if (!visited.add(node)) continue;
    for (String neighbour : graph.getOrDefault(node, List.of())) {
        stack.push(neighbour);
    }
}
```

Traversal order depends on neighbor order and stack insertion order. Do not promise deterministic output unless representation and algorithm deliberately provide it.

DFS as a Go-Deep-Then-Backtrack Image

Using the same graph and choosing the leftmost unvisited neighbor first:

```
path / stack evolution

A
A -> B
A -> B -> D
A -> B -> D -> G
backtrack to B
A -> B -> E
backtrack to A
A -> C -> F
complete
```

DFS is useful when the shape of a path matters: cycle detection, dependency exploration, connected components, and topological algorithms. An iterative implementation avoids call-stack overflow, but reproducing recursive entry/exit behavior may require stack frames containing both the vertex and next-neighbor index rather than a stack of vertices alone.

```
record Frame<V>(V vertex, Iterator<V> remaining) {}
```

This distinction matters for algorithms that perform work after all descendants complete. A simple pop-and-visit traversal is enough for reachability but not automatically enough for postorder computation.

Cycle Detection Requires More Than Visited

In a directed graph, an edge to any visited vertex does not necessarily indicate a cycle. DFS tracks three states:

```
WHITE = undiscovered
GRAY  = active on current DFS path
BLACK = completely processed

A(GRAY) -> B(GRAY) -> C(GRAY)
   ^                    |
   +--------------------+   edge to GRAY means a directed cycle

edge to BLACK means previously completed work, not a cycle by itself
```

For an undirected graph, the immediate parent edge must be ignored; otherwise every two-way edge appears to be a cycle. The algorithm follows the graph model.

Cycle detection is operationally useful for build dependencies, workflow definitions, service dependency configuration, and spreadsheet-like formulas. Reject cycles at configuration time when runtime execution assumes a DAG.

Topological Sorting for Dependency Order

A topological order lists every vertex after all of its prerequisites. It exists only for a DAG. Kahn's algorithm counts incoming edges, queues zero-indegree vertices, and removes their outgoing edges conceptually.

```
compile ----> test -----> package ----> deploy
   |                          ^
   +------> static-check -----+

initial indegree zero: [compile]
after compile:          [test, static-check]
possible order:         compile, test, static-check, package, deploy
```

```
Deque<String> ready = new ArrayDeque<>();
Map<String, Integer> indegree = calculateIndegrees(graph);
indegree.forEach((node, degree) -> {
    if (degree == 0) ready.addLast(node);
});

List<String> order = new ArrayList<>();
while (!ready.isEmpty()) {
    String node = ready.removeFirst();
    order.add(node);
    for (String next : graph.getOrDefault(node, List.of())) {
        if (indegree.compute(next, (k, d) -> d - 1) == 0) {
            ready.addLast(next);
        }
    }
}
if (order.size() != indegree.size()) {
    throw new CycleDetectedException();
}
```

Multiple valid orders may exist. Deterministic builds can use a priority queue for `ready` or stable neighbor order. If fewer than `V` vertices are emitted, a cycle prevents completion.

Connected Components

In an undirected graph, repeated BFS or DFS from each unvisited vertex identifies connected components.

```
component 1          component 2       isolated component
A -- B -- C          D -- E            F
```

```
List<Set<String>> components = new ArrayList<>();
Set<String> globallyVisited = new HashSet<>();

for (String vertex : graph.keySet()) {
    if (globallyVisited.add(vertex)) {
        Set<String> component = explore(vertex, graph, globallyVisited);
        components.add(component);
    }
}
```

For directed graphs, “connected” can mean weakly connected or strongly connected. Strongly connected components require reachability in both directions and algorithms such as Tarjan or Kosaraju. Name the

definition rather than saying only “find components.”

Dijkstra for Non-Negative Weighted Paths

BFS minimizes edge count. Dijkstra minimizes total weight when all edge weights are non-negative. It uses a priority queue ordered by the best known distance.

```
A --4--> B --2--> D
 \       ^
  1      |
   v     1
   C ----+

shortest A to D: A -> C -> B -> D, cost 1 + 1 + 2 = 4
not A -> B -> D, cost 6
```

```
record PathNode<V>(V vertex, long distance) {}

PriorityQueue<PathNode<String>> frontier = new PriorityQueue<>(
    Comparator.comparingLong(PathNode::distance));
Map<String, Long> distance = new HashMap<>();

distance.put(start, 0L);
frontier.add(new PathNode<>(start, 0L));

while (!frontier.isEmpty()) {
    PathNode<String> current = frontier.remove();
    if (current.distance() != distance.get(current.vertex())) continue;

    for (Edge<String> edge : weighted.outgoingFrom(current.vertex())) {
        long candidate = Math.addExact(current.distance(), edge.weight());
        if (candidate < distance.getOrDefault(edge.to(), Long.MAX_VALUE)) {
            distance.put(edge.to(), candidate);
            frontier.add(new PathNode<>(edge.to(), candidate));
        }
    }
}
```

Java's PriorityQueue has no decrease-key operation, so the common pattern inserts an improved record and ignores stale records when removed. Complexity with adjacency lists and a binary heap is typically O((V + E) log V). Negative weights invalidate Dijkstra's greedy assumption; choose an algorithm such as Bellman-Ford when negative weights are legitimate, and define negative-cycle behavior.

`Math.addExact` illustrates overflow awareness. Real path systems also define infinity representation, unreachable targets, tie-breaking, and whether weights can change during computation.

Minimum Spanning Trees Are Not Shortest-Path Trees

For a connected undirected weighted graph, a minimum spanning tree connects all vertices with minimum total edge weight. It does not necessarily provide the shortest route from a particular source.

```
original graph              one minimum spanning tree
A --1-- B                   A --1-- B
| \     |                   | \
4  2    3                   4  2
|    \  |                   |    \
C --5-- D                   C     D      (connect all, no cycle)
```

Kruskal sorts edges by weight and adds an edge when Union-Find says it joins different components. Prim grows a tree from a frontier using a priority queue. Choose from representation and workload. If the graph is disconnected, the result is a minimum spanning forest unless the API treats disconnection as failure.

Graph Algorithms in Production Systems

Graph terminology maps directly to common architecture problems:

```
problem                              graph interpretation
-------------------------------------------------------------------------
service startup                      dependency DAG + topological order
package/build dependencies           directed graph + cycle detection
social or fraud relationships        large sparse graph + neighborhoods
workflow reachability                directed traversal
network routing                      weighted shortest path
authorization inheritance            directed reachability with cycle policy
cluster connectivity                 components / Union-Find
job prerequisites                    DAG + ready queue
```

For graphs larger than memory, algorithms must consider partitioning, external storage, distributed traversal, and the cost of random access. A database query, graph database, search index, or stream processor may be more appropriate than loading every vertex into a Java Map.

Common Graph Traps

The most frequent mistakes are semantic rather than syntactic:

- Forgetting a visited set and looping forever on cycles.
- Marking visited too late and enqueuing the same vertex repeatedly.
- Forgetting reverse edges in an undirected representation.
- Treating an edge to any visited DFS vertex as a directed cycle.
- Using BFS for weighted shortest paths.
- Using Dijkstra with negative weights.
- Allowing mutable vertex keys to corrupt adjacency maps.
- Assuming traversal order is deterministic with HashMap/HashSet neighbors.
- Using recursion on an adversarially deep graph.
- Ignoring integer overflow when summing weights.
- Calling an evolving distributed relationship snapshot “the graph” without version semantics.

A Strong Interview Explanation for Graphs

Start by classifying the graph: directed or undirected, weighted or unweighted, sparse or dense, cyclic or acyclic. Choose representation from operations: adjacency lists for sparse traversal, sets for frequent edge tests, matrices for small dense graphs, and edge lists for edge-sorting algorithms.

Then connect algorithm to guarantee. BFS provides minimum edge count in unweighted graphs. DFS supports reachability, cycle reasoning, and postorder. Topological sort requires a DAG. Dijkstra requires non-negative weights. Union-Find answers incremental connectivity, while minimum spanning trees optimize total connection cost rather than source-to-target distance. State complexity in `V` and `E`, name memory requirements, and mention deterministic ordering and failure policy.

Tries and Prefix Search

A trie stores keys by successive characters or tokens. Lookup is O(L) in key length rather than the number of stored keys, and prefix traversal is natural.

```
root
  c -> a -> t
       \-> r
  d -> o -> g
```

Tries can consume substantial memory because each node has child structure and metadata. Compressed tries, sorted arrays, database indexes, or search engines may be better. Use them when prefix behavior and scale justify the representation.

Union-Find for Connectivity

Disjoint-set union tracks connected components with parent links. Path compression and union by rank or size make operations nearly constant amortized time.

```
int find(int x) {
    if (parent[x] != x) {
        parent[x] = find(parent[x]);
    }
    return parent[x];
}
```

It is effective for network connectivity, clustering, and Kruskal's minimum-spanning-tree algorithm. It answers connectivity under unions; it is not a general replacement for graph storage and does not naturally handle edge deletion.

Boundedness and Backpressure

Data structures also encode capacity policy. An unbounded queue accepts work faster than consumers can process it, converting overload into memory growth and latency.

```
producer rate 5,000/s -> unbounded queue -> consumer rate 3,000/s
backlog grows 2,000/s until memory or latency fails
```

A bounded BlockingQueue can reject, block, or trigger load shedding according to the executor policy. The bound should come from latency and capacity goals, not an arbitrary large number.

Similarly, every cache needs a size or weight policy, every deduplication set needs a retention window, and every batch needs a maximum cardinality. Boundedness is part of correctness in production.

Complexity Traps in Application Code

Nested use of linear collections can create accidental quadratic behavior:

```
for (Order order : orders) {
    if (blockedCustomerIdsList.contains(order.customerId())) {
        reject(order);
    }
}
```

If both collections contain n elements, repeated List.contains can approach O(n squared). Converting the membership side to a HashSet changes expected lookup to O(1), making the loop expected O(n), at the cost of building and storing the set.

Other traps include sorting repeatedly inside a loop, concatenating immutable strings in large loops, removing from the front of ArrayList, and recursively traversing an unbounded depth.

Production and Senior-Level Reasoning

Explain a choice with the full chain: operation profile, expected complexity, memory representation, ordering semantics, concurrency boundary, and failure behavior. For example: "I chose a bounded ArrayBlockingQueue because producers can exceed consumers; the bound caps retained work, and rejection activates load shedding before the JVM exhausts memory."

Monitor cardinality, queue depth and age, cache hit rate and eviction, distribution of key popularity, allocation rate, retained size, and tail latency. A theoretically suitable structure can still fail if the workload assumptions are wrong.

Interview-Ready Summary

Choose structures from dominant operations and invariants. Arrays and ArrayList serve indexed and sequential access; ArrayDeque serves stacks and queues; PriorityQueue serves repeated best-item retrieval; hash tables serve exact lookup; trees serve sorted and range operations; adjacency lists serve sparse graphs.

Complexity is a scaling model, not a stopwatch. Consider constants, locality, boxing, allocation, cardinality, and concurrency. Encode boundedness explicitly for queues, caches, sets, and batches. Explain alternatives and production failure modes rather than naming one collection in isolation.

Revision Notes

- Start with operations, ordering, cardinality, memory, and concurrency requirements.
- Big-O describes growth, not exact runtime.
- Arrays offer fixed contiguous storage and O(1) indexing.
- ArrayList is the normal dynamic indexed list.
- ArrayDeque is preferred for stacks and double-ended queues.
- PriorityQueue is a heap; iteration is not sorted.
- Heap insertion and head removal are O(log n).
- HashMap favors exact lookup; TreeMap favors ordering and ranges.
- LinkedHashMap can combine lookup with insertion or access order.
- A simple LRU is not automatically a production cache.
- Adjacency lists suit sparse graphs.
- BFS uses a queue and visits an unweighted graph in O(V + E).
- Mark BFS nodes visited when enqueuing.
- DFS recursion can overflow on deep graphs.
- Tries support prefix lookup but can use substantial memory.
- Union-find efficiently tracks connectivity under union operations.
- Unbounded queues turn overload into memory retention and latency.
- Repeated List.contains inside a loop can create quadratic behavior.
- Sets can convert repeated membership checks to expected constant time.
- A senior answer connects complexity to memory, concurrency, and operations.
- Validate assumptions with production cardinality and profiling.
- Classify graphs as directed/undirected, weighted/unweighted, sparse/dense, and cyclic/acyclic.
- Adjacency lists use O(V + E) space and usually fit sparse graphs.
- A matrix uses O(V squared) space but provides O(1) edge checks.
- BFS discovers minimum-edge paths in unweighted graphs.
- DFS supports reachability, cycle detection, postorder, and component discovery.
- Directed cycle detection distinguishes active GRAY vertices from completed BLACK vertices.
- Topological order exists only for a DAG; incomplete Kahn output indicates a cycle.
- Dijkstra requires non-negative edge weights and commonly uses a priority queue.
- Minimum spanning trees minimize total connection cost, not source-path distance.
- Traversal order is deterministic only when representation and neighbor ordering are deterministic.
- Mutable vertex keys can corrupt adjacency maps.
- For very large graphs, storage locality and partitioning can dominate Big-O reasoning.